

Hands-on session “Starting with pysteps”



Session overview

Time	Content	Speaker	Duration
09:00	Welcome / Course overview	Ruben	20'
09:20	Lecture: "Pysteps in operations"	Ruben, Lesley	40'
10:00	Coffee break		30'
10:30	Hands-on session "Starting with pysteps"	Jenna, Miguel, Mats	120'
12:30	Lunch break		60'
13:30	Hands-on session "Blending with NWP"	Mats, Lesley, Ruben	120'
15:30	Coffee break		30'
16:00	Lecture: "AI for nowcasting"	Lesley, Jenna, Ruben	45'
16:45	Discussion	All	15'
17:00	End of the short course		



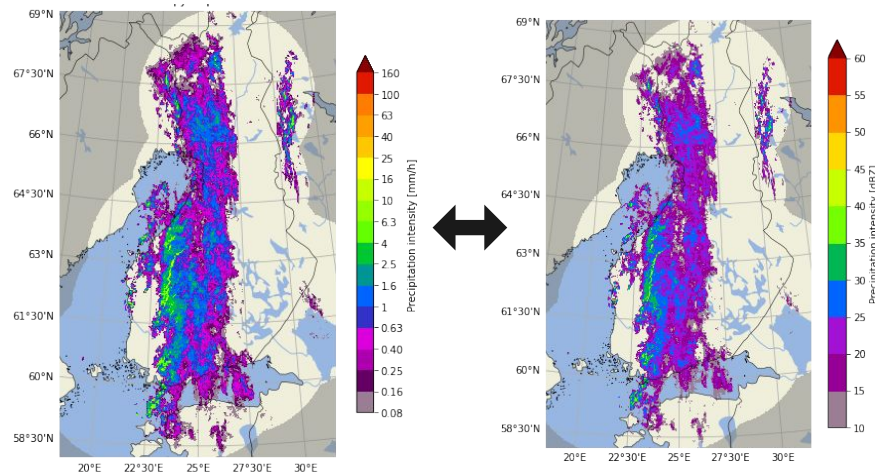
Outline

- The purpose of this session is to give a “hands-on” experience of using pysteps
- The session is divided into 5 exercise blocks + bonus exercises
 - In the morning, we focus on getting started with the fundamentals of pysteps
 - The afternoon will give an in-depth blending with NWP experience
- In the following slides, we give an introduction to the basic pysteps workflow

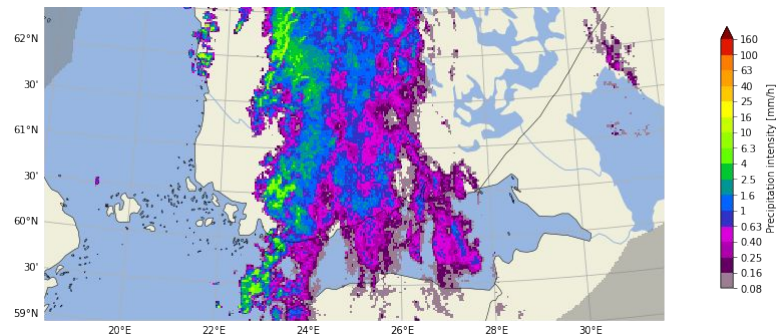
Input data

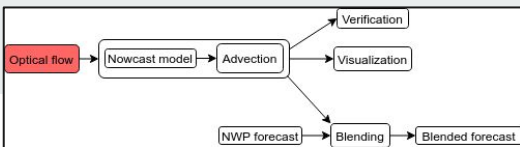
- pysteps uses radar composites as the main input data source
- Reading input data from various sources has been implemented in the **io.importers** module
- To implement your own importers, you can use cookiecutter:
https://pysteps.readthedocs.io/en/stable/developer_guide/importer_plugins.html
- The **utils** module contains different methods for transforming, clipping and upsampling the input data

Transformations and conversions between units



Clipping

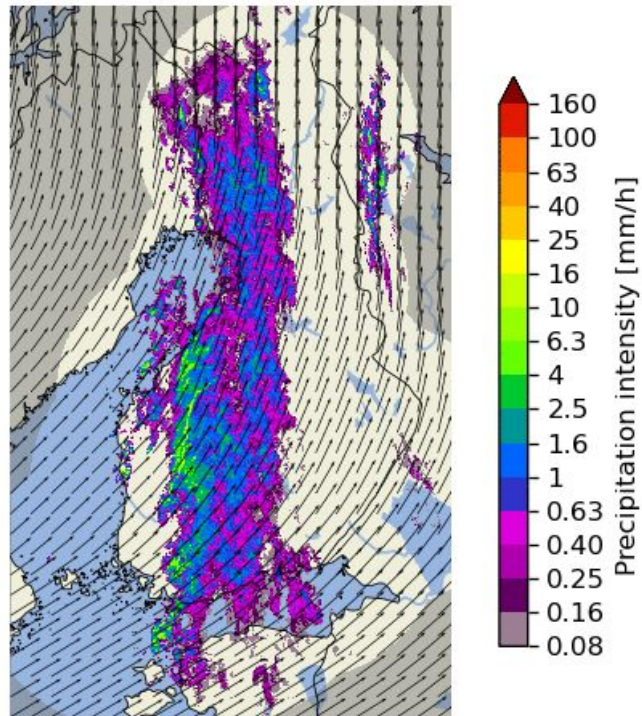


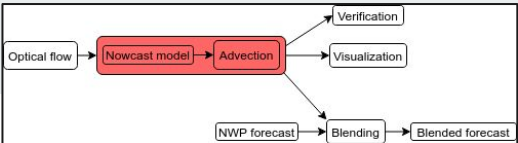


Optical Flow and Extrapolation

- Advection (optical flow & extrapolation) is the key component of all pysteps nowcasting methods
- All methods are based on the “Lagrangian persistence” nowcast shown on the right
- Three different types of optical flow methods have been implemented in the **motion** module:
 - feature tracking: Lucas-Kanade
 - variational: VET and Proesmans
 - spectral: DARTS
- For advection, pysteps implements the backward semi-Lagrangian scheme in the **extrapolation** module

2016-09-28 15:50:00 + 5 min

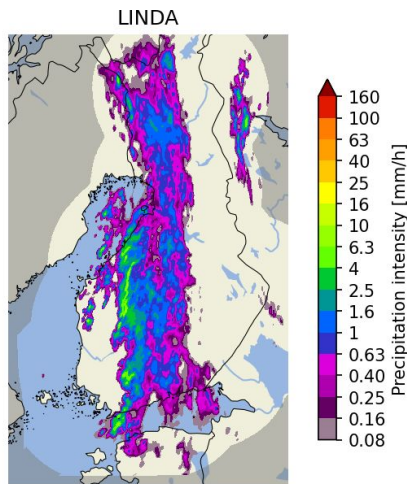
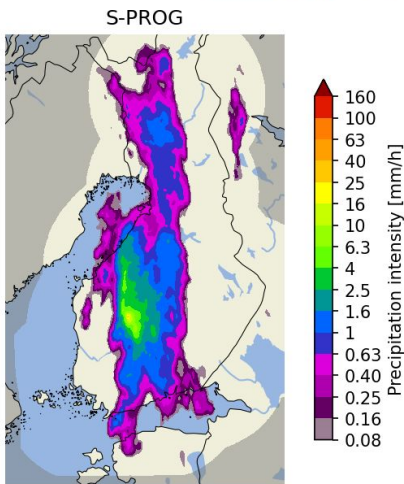
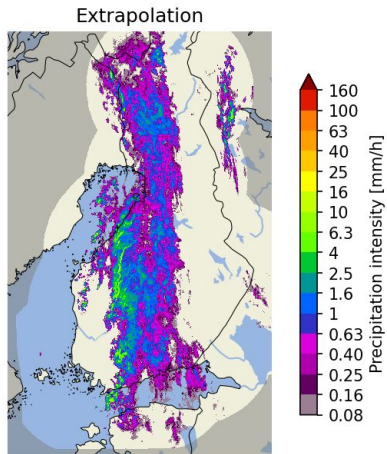




Deterministic Nowcasts

The main methods implemented in the **nowcasts** module:

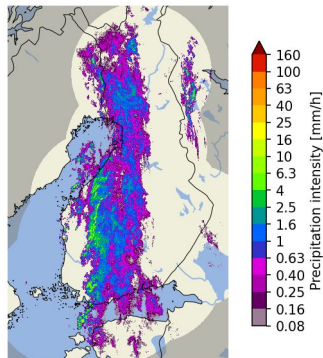
Method	Pros	Cons	Typical computation time
Extrapolation	very fast	no prediction of growth or decay of precipitation	< 10 seconds
S-PROG	- for low-intensity precipitation (< 1-2 mm/h) has generally the best skill - choose for stratiform events	inability to preserve the spatial structure of rainfall fields, and particularly convective cells	< 20 seconds
LINDA-D	- the most accurate method for intense precipitation (> 1-2 mm/h) - choose for convective events	slow to compute	might take several minutes



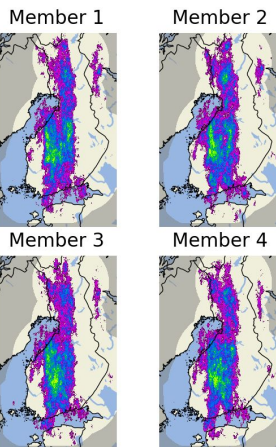
Ensemble Nowcasts

- The main ensemble methods implemented in the **nowcasts** module are STEPS and LINDA-P
- They model two sources of uncertainty: advection field estimation and Lagrangian growth and decay
- The basic rule for choosing the method:
 - stratiform events: STEPS
 - convective events: LINDA-P
- LINDA-P generally produces more realistic ensemble members
- Computation times for the 4-member ensembles shown on the right:
 - STEPS: ~20 seconds
 - LINDA-P: ~5 minutes

Observations at
2016-09-28 15:50 UTC

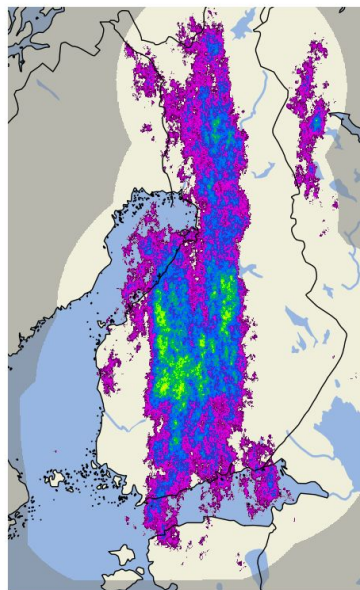


Nowcast ensemble

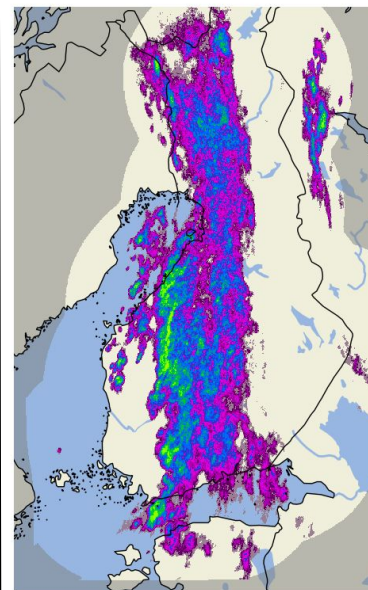


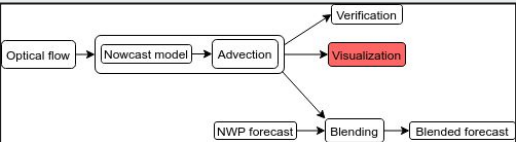
First ensemble members

STEPS



LINDA-P



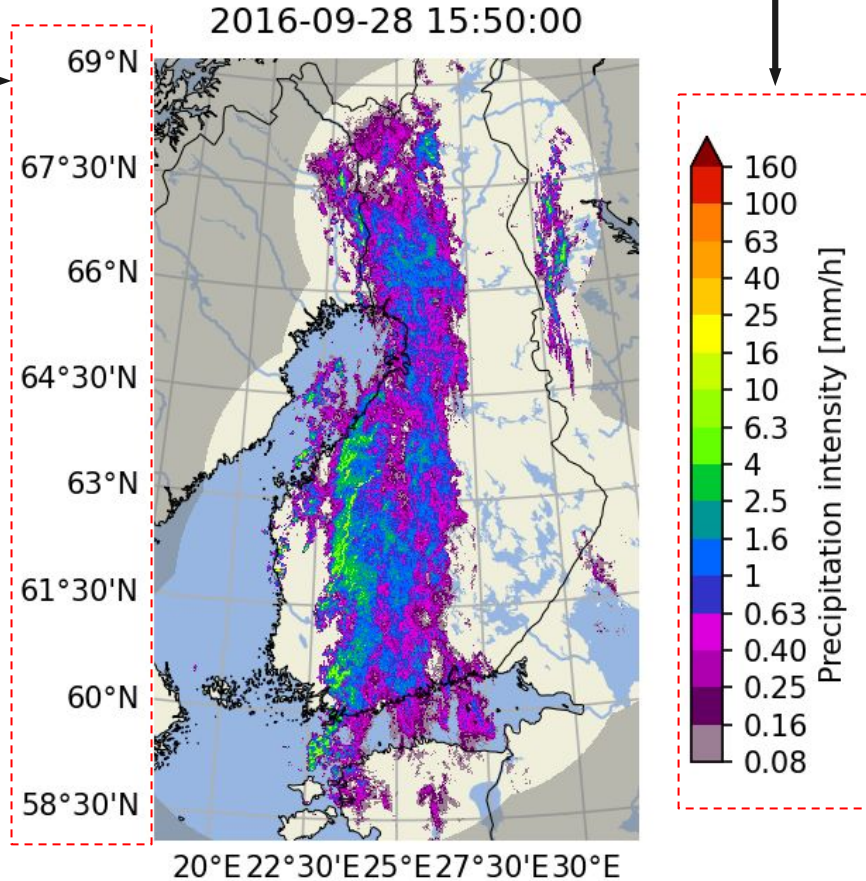
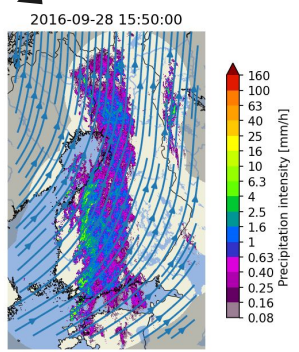
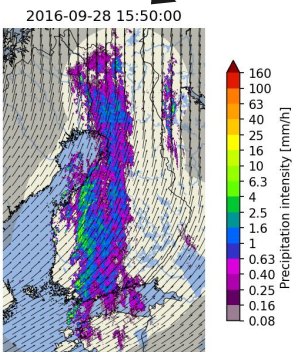


Colorbars with several pre-configured scales and for different data units

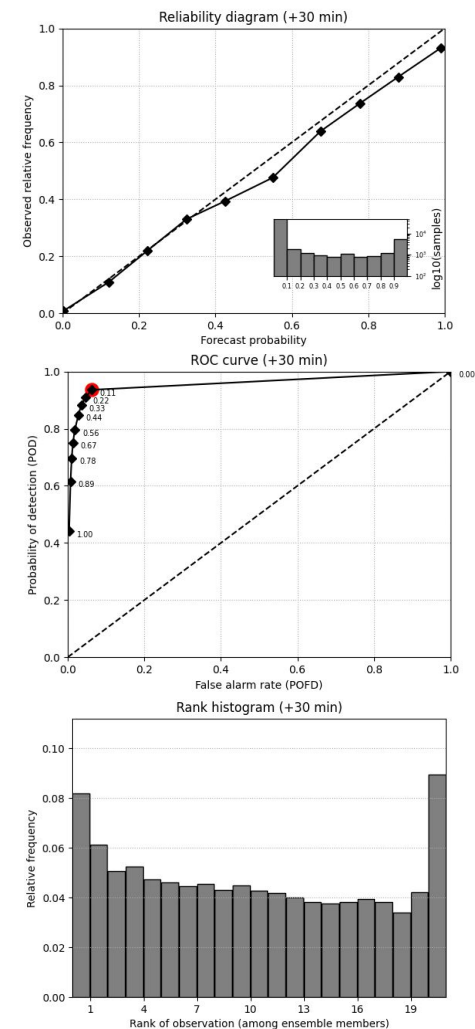
Longitude-latitude lines with labels

Visualization tools

- Extensive set of visualization tools has been implemented in the **visualization** module
- Support for multiple layers: basemap, precipitation and motion field:
 - plotting of basemaps by using cartopy
 - quivers and streamlines for advection fields



Examples of verification plots for 30-minute STEPS nowcasts

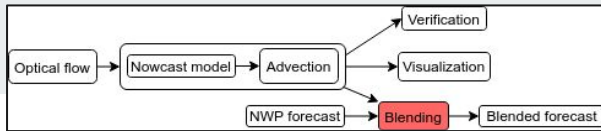


Verification tools & metrics

- A large number of verification utilities and metrics have been implemented in the **verification** module
- Functionality
 - creation of verification objects and aggregation from multiple nowcasts
 - plotting of verification results

Metrics

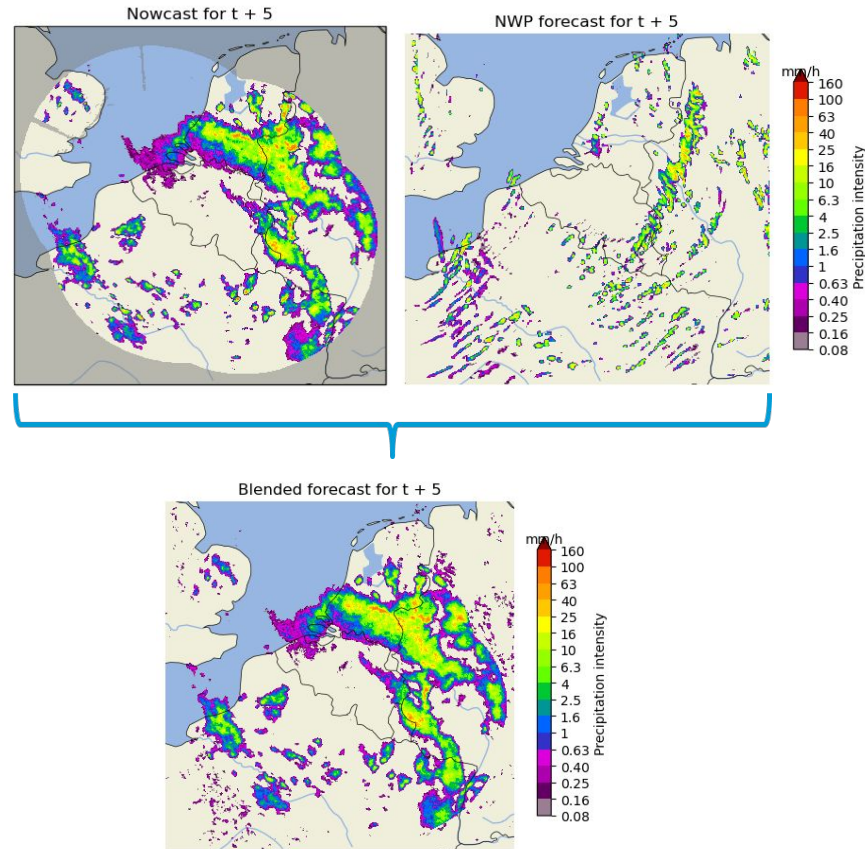
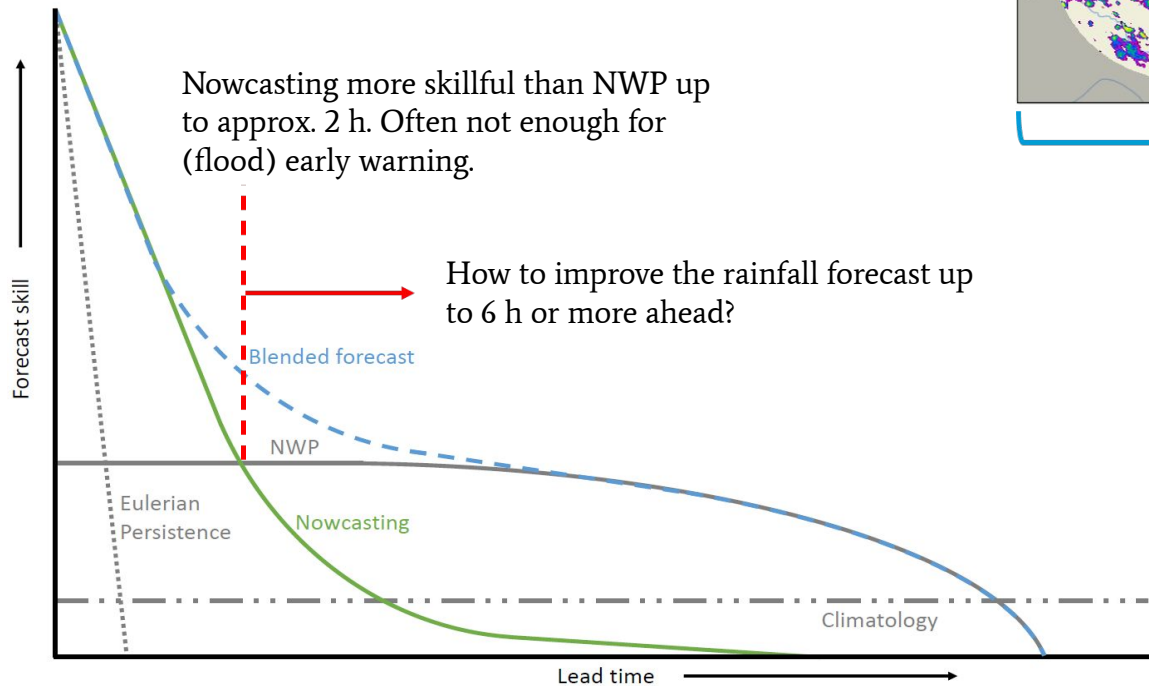
- Deterministic
 - categorical: CSI, ETS, POD, FAR
 - continuous: MAE, ME
 - scale/intensity-based metrics: FSS, intensity-scale
 - radially averaged power spectral density (RAPSD)
- Probabilistic
 - CRPS
 - reliability diagram
- Ensemble
 - spread
 - rank histogram



Blending with NWP

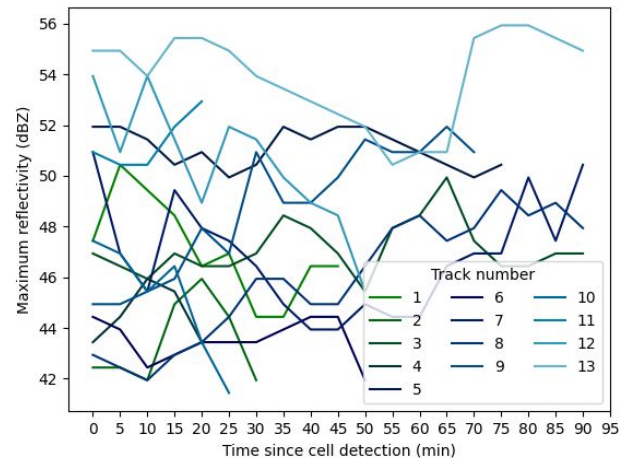
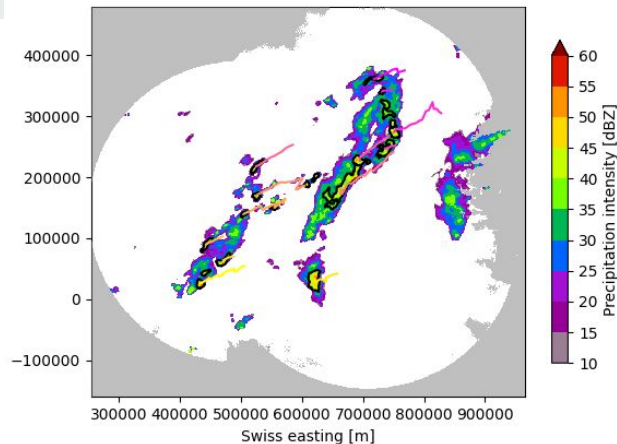
Nowcasting more skillful than NWP up to approx. 2 h. Often not enough for (flood) early warning.

How to improve the rainfall forecast up to 6 h or more ahead?



T-DaTing module: Thunderstorm Detection and Tracking

- Identify and track thunderstorm cells from radar images
- Visualize cells and tracks in time
- Study properties of the cell tracks
- Tracking algorithm from the Swiss TRT Thunderstorms Radar Tracking algorithm
 - Hering et al., 2004, ERAD 2004
 - Feldmann et al., 2021, Weather Clim. Dynam
- For how to use, see the [example](#) in the gallery

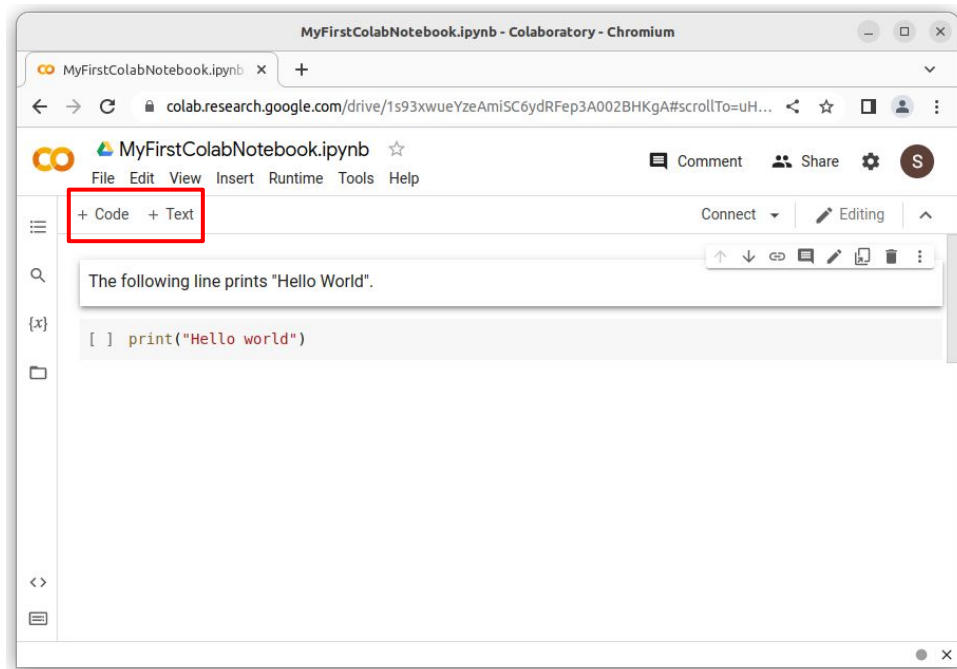


Getting started with the exercises

Introduction to Google Colab

<https://research.google.com/colaboratory>

- Colab is a web-based Python environment
- Free of charge to use: you only need to have a Google account
- Support for different types of blocks:
 - *Code*: runnable code with output
 - *Section*: for organizing your notebook
 - *Text*: descriptions of code blocks
- The default environment has:
 - Python 3.7 with a large number of scientific packages pre-installed
 - 1 Intel Xeon CPU core with 2 threads
 - 12 GB memory, 100 GB disk space + Google drive





Exercise 1: Install and configure pysteps in Colab

1. Install pysteps dependencies by using pip or conda-colab & mamba, see environment.yml
2. Install pysteps by using pip & git
3. Download example data to Google drive
4. Create pystepsrc and configure pysteps



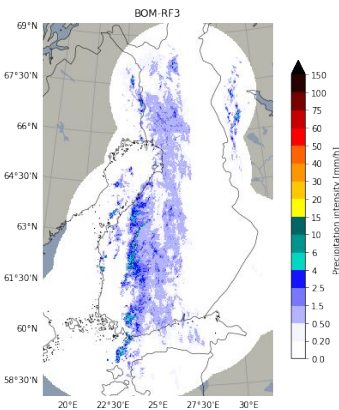
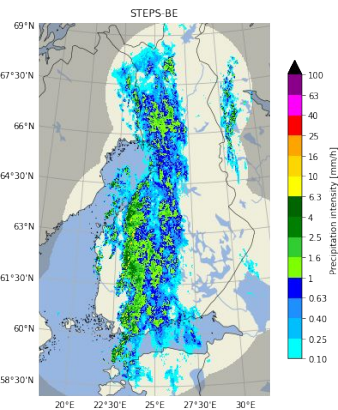
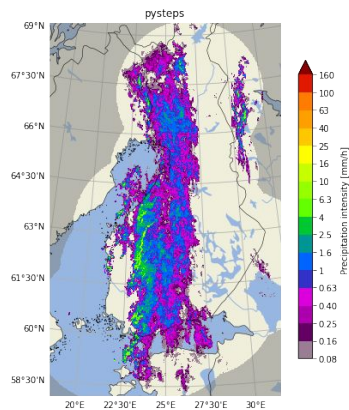
Exercise 2: Read, visualize and process input data

1. Install pysteps with helper_setup.ipynb notebook
2. Read the example data downloaded above, pick either fmi or mch dataset
3. Plot precipitation fields on top of a basemap
4. Try different preprocessing methods:
 - a. upsampling
 - b. clipping
 - c. unit conversions
 - d. Logarithmic transformation
 - e. normal quantile transformation

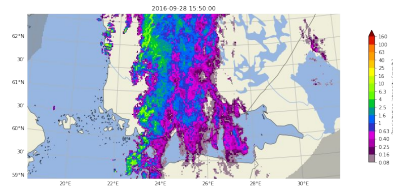
Exercise 2: Read, visualize and process input data

Basemap + precipitation with different colormaps

2016-09-28 15:50:00

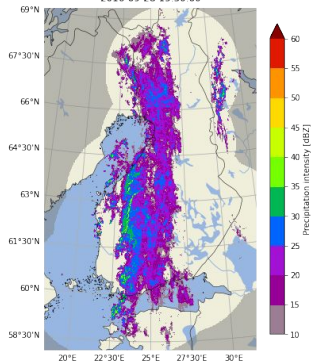


Clipping

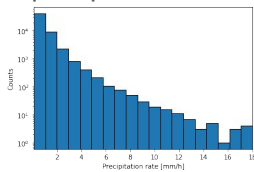


Conversion:
rain rate <-> reflectivity

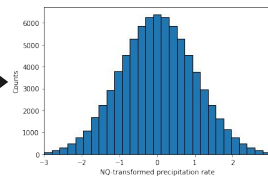
2016-09-28 15:50:00



Distribution of
precipitation rates



After NQ
transformation





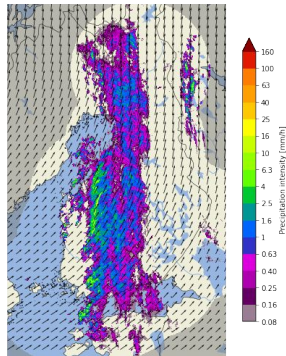
Exercise 3: Optical flow and extrapolation

1. Load example data with `helper_input_data.ipynb` notebook
2. Try different optical flow methods: Lucas-Kanade, VET and DARTS.
Questions:
 - a. What the advection fields look like? Which one would you trust?
 - b. What is the effect of data transformations (e.g. log-transform) before applying the optical flow?
3. Apply semi-Lagrangian extrapolation and plot the extrapolated precipitation fields for lead times of 15, 30, 45 and 60 minutes
4. Apply extrapolation with time steps that are different from the input data

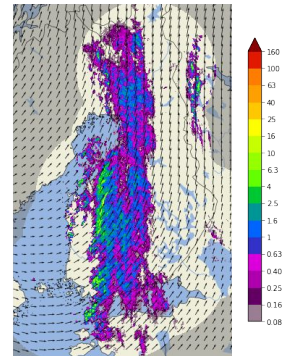
Exercise 3: Optical flow and extrapolation

Different optical flow methods

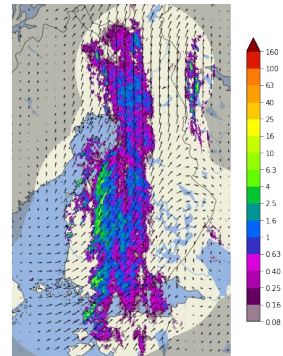
Lucas-Kanade



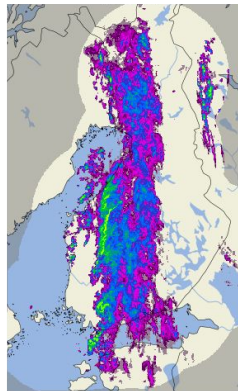
VET



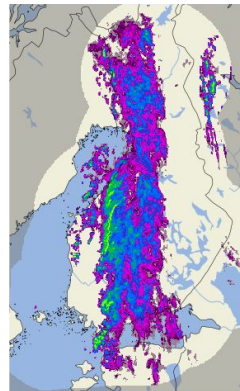
DARTS



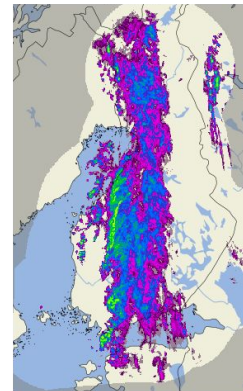
15 minutes



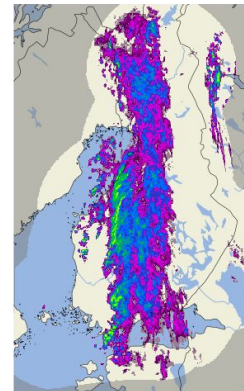
30 minutes



45 minutes



60 minutes



Extrapolation



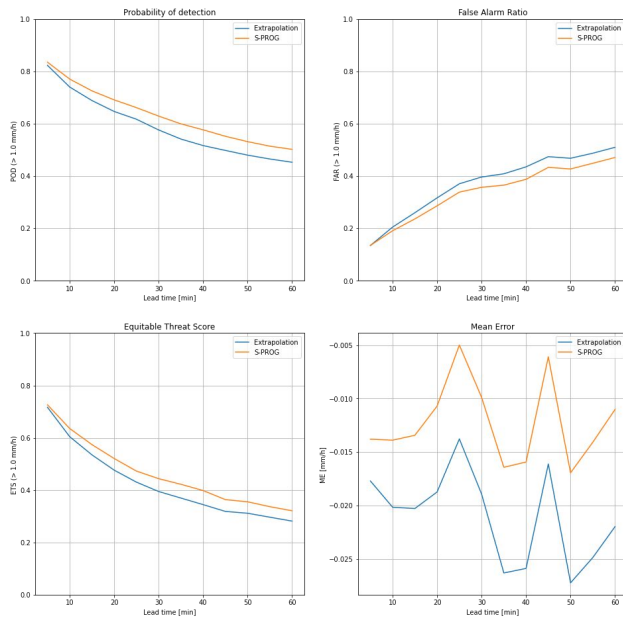
Exercise 4: Deterministic nowcasting

1. Load example data with
`helper_nowcasting_methods.ipynb` notebook
2. Create & visualize nowcasts with at least
extrapolation method
3. Calculate verification metrics between
observations and nowcasts
4. Export the created nowcast to NetCDF file

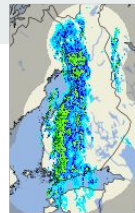
Deterministic nowcasting methods in
pysteps:

- Extrapolation
- ANVIL
- LINDA
- S-PROG

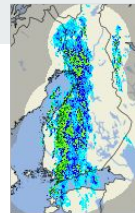
Exercise 4: Deterministic nowcasting



Observation at +15 minutes



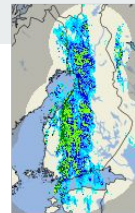
Observation at +30 minutes



Observation at +45 minutes



Observation at +60 minutes



Extrap. nowcast +15 minutes



Extrap. nowcast +30 minutes



Extrap. nowcast +45 minutes



Extrap. nowcast +60 minutes



S-PROG. nowcast +15 minutes



S-PROG. nowcast +30 minutes



S-PROG. nowcast +45 minutes



S-PROG. nowcast +60 minutes



Remember to check parameter values for each nowcasting method (default values not necessarily good for all datasets)!



Exercise 4: Probabilistic nowcasting

1. Load example data with
`helper_nowcasting_methods.ipynb` notebook
2. Create an ensemble STEPS nowcasts with 20 ensemble members and 12 lead times (+1 h)
3. Visualize output, ensemble members and probabilities
4. Calculate probabilistic verification metrics between observations and nowcasts
5. If time allows: Try out a LINDA-P nowcast

Probabilistic nowcasting methods in pysteps:

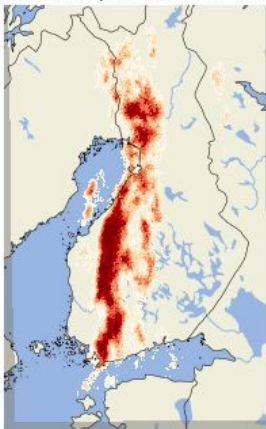
- Lagrangian probability
- STEPS
- LINDA-P

Exercise 4: Probabilistic nowcasting

Exceedance prob. at +15 minutes



Exceedance prob. at +30 minutes



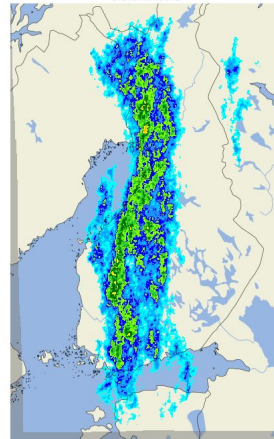
Exceedance prob. at +45 minutes



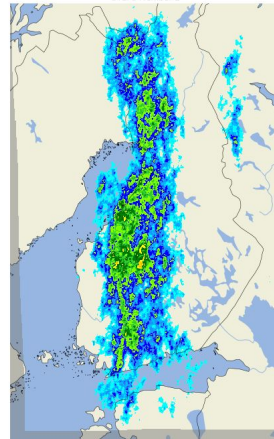
Exceedance prob. at +60 minutes



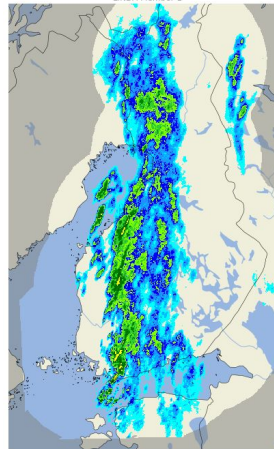
STEPS Member 1



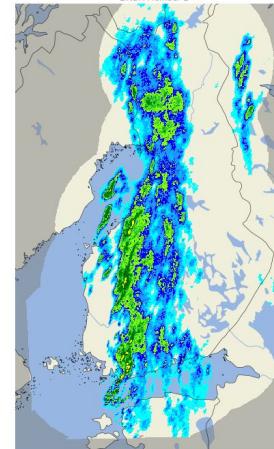
STEPS Member 2



LINDA Member 1



LINDA Member 2



Summary and some extra information

Benchmarking other methods



- When selecting correct method to benchmark against e.g. machine learning models, consider what do you want to achieve with the model
 - preserve variance
 - minimize error
 - represent uncertainty?
- To get comparable forecasts, use methods that try to achieve similar objectives
- For a more detailed discussion, see [the documentation](#)

From the documentation:

Nowcast type	Machine learning	pysteps	Verification
Deterministic (variance-preserving)	SRGAN, Others?	pysteps.nowcasts.extrapolation (any optical flow method)	MSE, RMSE, MAE, ETS, etc
Deterministic (error-minimization)	Classical ANNs, (deep) CNNs, random forests, AdaBoost, etc	pysteps.nowcasts.sprog , pysteps.nowcasts.anvil or ensemble mean of pysteps.nowcasts.steps/linda	MSE, RMSE, MAE, ETS, etc or better normalized scores, etc
Probabilistic (quantile-based)	Quantile ANN, quantile random forests, quantile regression	pysteps.nowcasts.lagrangian_probability or probabilities derived from pysteps.nowcasts.steps/linda	Reliability diagram (predicted vs observed quantile), probability integral transform (PIT) histogram
Probabilistic (ensemble-based)	GANs, VAEs, etc	Ensemble and probabilities derived from pysteps.nowcasts.steps/linda	Probabilistic verification: reliability diagrams, continuous ranked probability scores (CRPS), etc. Ensemble verification: rank histograms, spread-error relationships, etc